

```
10 20:20:29.674932 IP (tos 0x8, ttl 64, id 57681, offset 0, flags
    [DF], proto 6, length: 288)
    10.0.0.123.1027 > 169.229.62.97.22: . [tcp sum ok]
    1431:1667(236) ack 2105

11 20:20:29.675143 IP (tos 0x8, ttl 64, id 57682, offset 0, flags
    [DF], proto 6, length: 288)
    10.0.0.123.1027 > 169.229.62.97.22: . [tcp sum ok]
    1667:1903(236) ack 2105
```

In the `tcpdump` output, the connection has already been set up and MSS options have been exchanged. All packets on the connection have the *DF* bit field set, so both ends are performing PMTUD. The remote side's first packet is 588 bytes long, which transitions the router successfully in one piece, despite our configuration of the MTU on the PPPoE links being 288 bytes. The reason for this is asymmetry in the MTU configuration. Although the local end of the PPPoE link is using a maximum *transmission* unit of 288 bytes, the other end is using a larger size SMSS, presumably 1492 bytes. This leaves us in the situation where our outgoing packets need to be small (288 bytes or less), and packets traveling in the reverse direction can be larger.

When the local end attempts to send a larger packet of size 588 bytes with the *DF* bit field turned on, a PTB message is generated by the router (10.0.0.1), indicating that the appropriate MTU for the next-hop link is 288 bytes. The TCP responds by sending its next packet with size 288 bytes, as instructed. To then send the rest of the sequence numbers it attempted to send in its 588-byte packet, it sends two additional packets, of sizes 288 and 116. We see a similar pattern of sizes repeats during the course of the file transfer.

The PMTU discovery process is one of the only ways TCP explicitly attempts to adapt its segment size after a connection has started, at least when large amounts of data are transferred. The size of a segment can affect the overall throughput performance, as can the window size. We discuss how these affect overall performance in Chapter 15.

13.5 TCP State Transitions

We have described numerous rules regarding the initiation and termination of a TCP connection, and we have seen which types of segments are sent during different phases of a connection. The rules that determine what TCP does are determined by what state TCP is in. The current state is changed based on various stimuli, such as segments that are transmitted or received, timers that expire, application reads or writes, or information from other layers. These rules can be summarized in TCP's state transition diagram.

13.5.1 TCP State Transition Diagram

TCP's state transition diagram is shown in Figure 13-8. States are indicated by ovals and transitions between states by arrows. Each endpoint of a connection transitions through the states. Some transitions are triggered by the receipt of a segment with certain control bit fields set (e.g., *SYN*, *ACK*, *FIN*). Some transitions

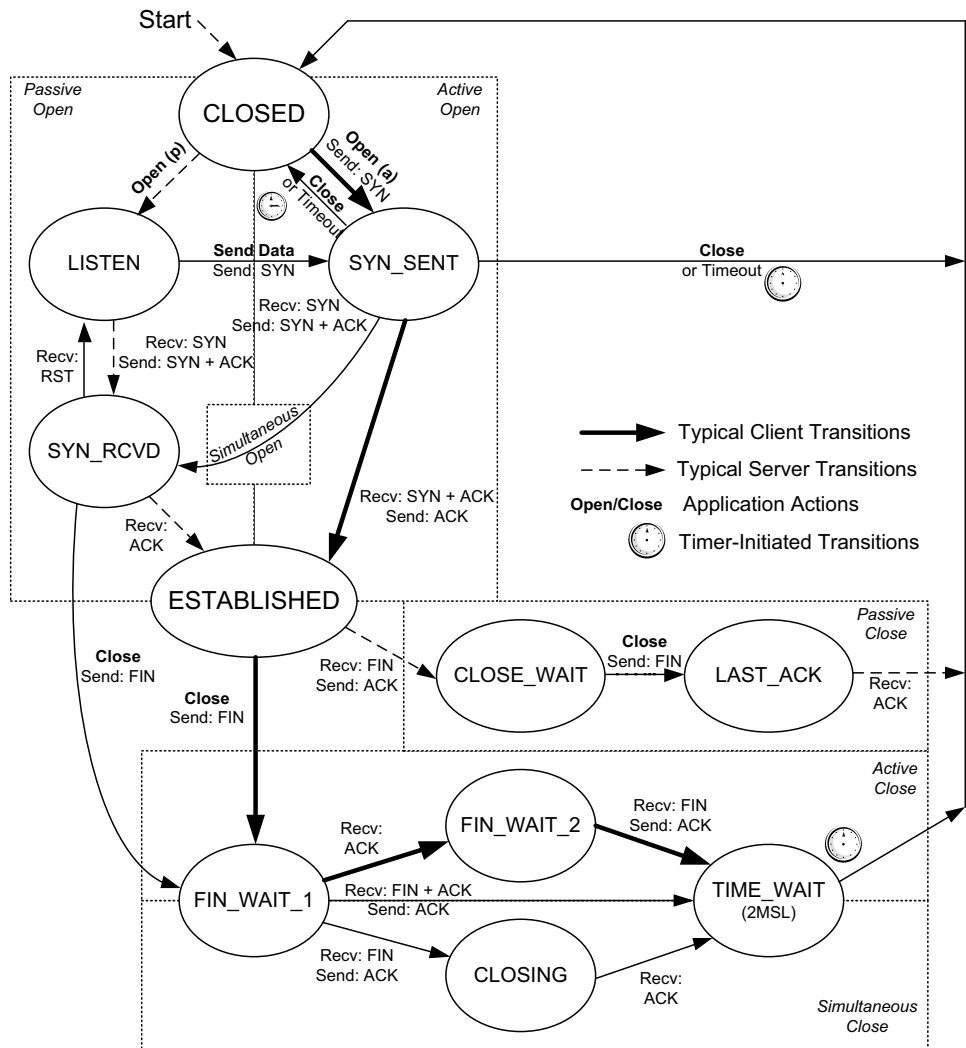


Figure 13-8 The TCP state transition diagram (also called finite state machine). Arrows represent transitions between states due to segment transmission, segment reception, or timers expiring. The bold arrows indicate typical client behavior, and the dashed arrows indicate typical server behavior. The boldface directives (e.g., open, close) are actions performed by applications.

also cause a segment with particular control bit fields set to be sent. Other transitions may be triggered by application actions or by timers expiring. Each of these cases is indicated in the diagram as a textual annotation near the associated transition arrow. When initialized, TCP starts in the CLOSED state. Usually an immediate transition takes it to either the SYN_SENT or LISTEN state, depending on whether the TCP is asked to perform an active or passive open, respectively.

Note in this diagram that only a subset of the state transitions is “typical.” We have marked the normal client transitions with a darker solid arrow, and the normal server transitions with a dashed arrow. The two transitions leading to the ESTABLISHED state correspond to opening a connection, and the two transitions leading from the ESTABLISHED state are for the termination of a connection. The ESTABLISHED state is where data transfer can occur between the two ends in both directions. Chapters 14–17 describe what happens in this state.

We have labeled the FIN_WAIT_1, FIN_WAIT_2, and TIME_WAIT states as being (at least partially) in a box called “Active Close.” These are the set of states entered when the local application initiates a close request. Two other states (CLOSE_WAIT and LAST_ACK) are collected in a dashed box with the label “Passive Close.” These states correspond to waiting for a peer to acknowledge a FIN segment and perform its close. Simultaneous close, which is a form of double active close, uses the CLOSING state.

The names of the 11 states (CLOSED, LISTEN, SYN_SENT, etc.) in this figure are based on the names output by the `netstat` command in UNIX, Linux, and Windows, which are themselves based on the names originally used in [RFC0793]. The state CLOSED is not really an “official” state but has been added as a useful starting point and ending point for the diagram.

The state transition from LISTEN to SYN_SENT is legal in the TCP protocol but is not supported by Berkeley sockets and is rarely seen. The transition from SYN_RCVD back to LISTEN is valid only if the SYN_RCVD state was entered from the LISTEN state (the normal scenario), not from the SYN_SENT state (a simultaneous open). This means that if we perform a passive open (enter LISTEN), receive a SYN, send a SYN with an ACK (enter SYN_RCVD), and then receive a reset instead of an ACK, the endpoint returns to the LISTEN state and waits for another connection request to arrive.

Figure 13-9 shows the normal TCP connection establishment and termination, detailing the different states through which the client and server pass. It is a simpler version of Figure 13-1 showing the relevant states but not the options or ISN details. We assume in Figure 13-9 that the client on the left side does an active open and the server on the right side does a passive open. Although we show the client doing the active close, as we mentioned earlier, either side can do the active close.

13.5.2 TIME_WAIT (2MSL Wait) State

The TIME_WAIT state is also called the 2MSL wait state. It is a state in which TCP waits for a time equal to twice the *Maximum Segment Lifetime* (MSL), sometimes called *timed wait*. Every implementation must choose a value for the MSL. It is

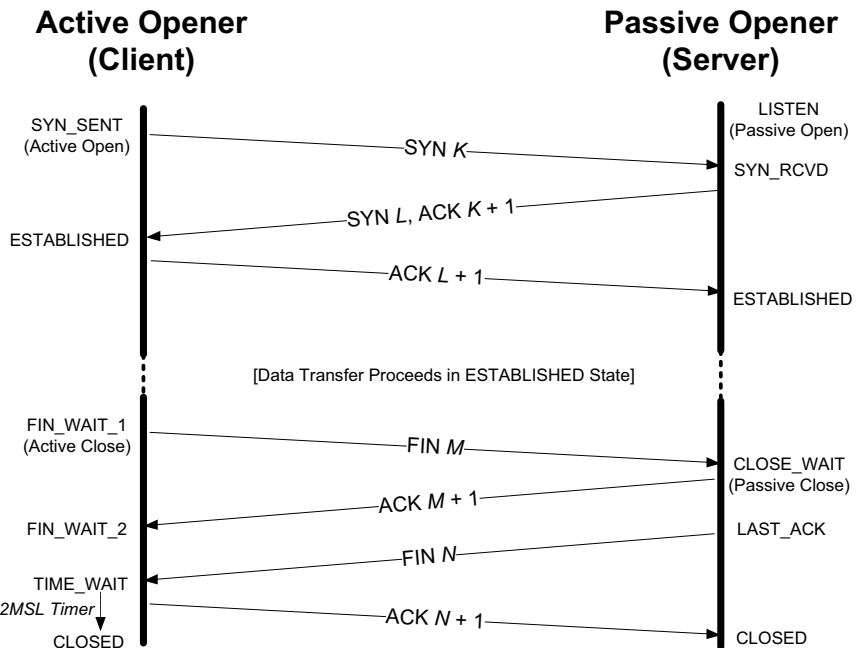


Figure 13-9 TCP states corresponding to normal connection establishment and termination

the maximum amount of time any segment can exist in the network before being discarded. We know that this time limit is bounded, because TCP segments are transmitted as IP datagrams, and the IP datagram has the *TTL* field or *Hop Limit* field that limits its effective lifetime (see Chapter 5). [RFC0793] specifies the MSL as 2 minutes. Common implementation values, however, are 30s, 1 minute, or 2 minutes. In most cases, the value can be modified. On Linux, the value `net.ipv4.tcp_fin_timeout` holds the 2MSL wait timeout value (in seconds). On Windows, the following registry key:

```
HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\TcpTimedWaitDelay
```

holds the timeout. It is permitted to be in the range of 30 to 300s. For IPv6, replace the term `Tcpip` with `Tcpip6`.

Given the MSL value for an implementation, the rule is: When TCP performs an active close and sends the final ACK, that connection must stay in the `TIME_WAIT` state for twice the MSL. This lets TCP resend the final ACK in case it is lost. The final ACK is resent not because the TCP retransmits ACKs (they do not consume sequence numbers and are not retransmitted by TCP), but because the other side will retransmit its FIN (which does consume a sequence number). Indeed, TCP will always retransmit FINs until it receives a final ACK.

Another effect of this 2MSL wait state is that while the TCP implementation waits, the endpoints defining that connection (client IP address, client port number, server IP address, and server port number) cannot be reused. That connection can be reused only when the 2MSL wait is over, or when a new connection uses an ISN that exceeds the highest sequence number used on the previous instantiation of the connection [RFC1122], or if the use of the Timestamps option allows the disambiguation of segments from a previous connection instantiation to not otherwise be confused [RFC6191]. Unfortunately, some implementations impose a more stringent constraint. In these systems, a local port number cannot be reused while that port number is the local port number of *any* endpoint that is in the 2MSL wait state on the system. We will see examples of this constraint in Listings 13-3 and 13-4.

Most implementations and APIs provide a way to bypass this restriction. With the Berkeley sockets API, the `SO_REUSEADDR` socket option enables the bypass operation. It lets the caller assign itself a local port number even if that port number is part of some connection in the 2MSL wait state. We will see, however, that even with this bypass mechanism for one socket (address, port number pair), the rules of TCP still (should) prevent this port number from being reused by another instantiation of the same connection that is in the 2MSL wait state. Any delayed segments that arrive for a connection while it is in the 2MSL wait state are discarded. Because the connection defined by the address/port 4-tuple in the 2MSL wait state cannot be reused during this time period, when a valid connection is finally established, we know that delayed segments from an earlier instantiation of this connection cannot be misinterpreted as being part of the new connection.

For interactive applications, it is normally the client that does the active close and enters the `TIME_WAIT` state. The server usually does the passive close and does not go through the `TIME_WAIT` state. The implication is that if we terminate a client, and restart the same client immediately, that new client cannot reuse the same local port number. This is not ordinarily a problem, because clients normally use ephemeral ports assigned by the operating system and do not care what the assigned port number is. (Recall, it is actually a recommended practice for them to be randomized for security reasons [RFC6056].) This is important to know because a client that makes a large number of connections quickly (especially to the same server) could conceivably have to delay while other connections terminate if ephemeral ports are in short supply.

With servers, however, the situation is different. They almost always use well-known ports. If we terminate a server process that has a connection established and immediately try to restart it, the server cannot assign its assigned port number to its endpoint (it gets an “Address already in use” binding error), because that port number is part of a connection that is in a 2MSL wait state. It may take from 1 to 4 minutes for the server to be able to restart, depending on the local system’s value for the MSL. We can see this scenario using our `sock` program. In Listing 13-3 we start the server, connect to it from a client, and then terminate the server.

Listing 13-3 A TCP connection must complete a 2MSL delay in the TIME_WAIT state before a port number can be reused by another process.

```
Linux% sock -v -s 6666
```

(now a client on another computer connects to this server)

```
connection on 192.168.10.144.6666 from 192.168.10.140.2623
```

(server stopped by typing interrupt character)

(now server is restarted)

```
Linux% sock -v -s 6666
```

```
can't bind local address: Address already in use
```

```
Linux% netstat -n -t
```

Active Internet connections (w/o servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	192.168.10.144:6666	192.168.10.140:2623	TIME_WAIT

(wait one minute and restart server again)

```
Linux% sock -v -s 6666
```

When we try to restart the server, the program outputs an error message indicating that it cannot bind its port number because the address is already in use. This really means that the address and port number combination is already in use; it is in a 2MSL wait state because of the previous connection. This is the more stringent restriction on port number reuse mentioned before. The output from the `netstat` command shows that the connection is in the TIME_WAIT state. Although clients do not typically experience as many issues with 2MSL wait states as servers do, we can demonstrate the same issue by having the client specify its own port number, as shown in Listing 13-4.

Listing 13-4 A client cannot reuse a port number while it is still being used by another connection in the 2MSL wait state.

(start server in one window)

```
Linux% sock -s -v 6666
```

(connect to it from another window)

```
Linux% sock -v 127.0.0.1 6666
```

(server identifies incoming connection)

```
connection on 127.0.0.1.6666 from 127.0.0.1.2091
```

(client identifies connection establishment, and is interrupted)

```
connected on 127.0.0.1.2091 to 127.0.0.1.6666
```

```
^C
```

(server identifies connection has terminated and exits)

```
connection closed by peer
```

```
Linux%
```

(client is restarted, specifying same port number as before)

```
Linux% sock -b 2091 -v 127.0.0.1 6666  
bind() error: Address already in use
```

(wait 30 seconds and try again)

```
Linux% sock -b 2091 -v 192.168.10.144 6666  
connect() error: Connection refused
```

The first time we execute the client we specify the `-v` option to see what the local (ephemeral) port number assigned to the client is (2091). The second time we execute the client we specify the `-b` option, telling the client to assign itself 2091 as its local port number instead of being given another ephemeral port number by the operating system. As we expect, the client cannot do this, because port 2091 is part of a connection that is in a 2MSL wait state. Once the wait is over (1 minute on this Linux machine), the client attempts to connect again, but the server exited when the connection was interrupted the first time, so it is refused. We shall see how TCP reset segments are used to signal this connection refused condition in Section 13.6.

We mentioned earlier that most systems provide a way of overriding the default behavior, which allows processes to bind to ports even if those ports are part of connections in the 2MSL wait state. Now we try the same scenario as before, but using the `-A` option to `sock`, which enables the bypass mechanism:

```
Linux% sock -A -v -s 6666  
Linux% sock -A -v -s 6666
```

In this example, we start the server with the `-A` option, which enables the `SO_REUSEADDR` socket option that we mentioned. By doing this, we allow the server to bind to its port even though it is part of a connection in the 2MSL wait state. If we try to use the client right away with the same port, however, the following happens:

```
Linux% sock -b 32840 -v 127.0.0.1 6666  
bind() error: Address already in use
```

Once again, the endpoint 127.0.0.1.32840 is in use, so the client fails. If, however, we also use the `-A` option for the client, we can force the connection to work:

```
Linux% sock -A -b 32840 -v 127.0.0.1 6666  
Connected on 127.0.0.1.32840 to 127.0.0.1.6666  
TCP_MAXSEG = 16383
```

Here we see that even though the same connection (4-tuple) is being used again before the 2MSL wait state expires, the use of the `-A` option has forced the connection to be allowed. Of course, this is all taking place on the same computer, so the operating system is able to ascertain what processes represent what ends

of the connections in the 2MSL wait state and (potentially, at least) keep them separate. What if we try the same thing again but establish the connection from another host? Here we test this idea:

(start server on first machine)

```
Linux% sock -v -s 6666
```

(connect to it from second - Windows - machine)

```
C:\> sock -A -v 10.0.0.1 6666
```

(server identifies incoming connection)

```
connection on 10.0.0.1.6666 from 10.0.0.3.2172
```

(client identifies connection establishment, and is interrupted)

```
connected on 10.0.0.3.2172 to 10.0.0.1.6666
```

```
^C
```

```
C:\>
```

(server identifies connection has terminated and exits)

```
connection closed by peer
```

```
Linux%
```

(client is restarted, specifying same port number as before)

```
C:\> sock -A -b 2091 -v 10.0.0.1 6666
```

```
connect() error: Address already in use
```

```
C:\> sock -A -b 2091 -v 10.0.0.1 6666
```

```
connect() error: Address already in use
```

(wait 30 seconds and try again)

```
C:\> sock -A -b 2091 -v 10.0.0.1 6666
```

```
connect() error: Connection refused
```

This example is similar to the previous one, except the client and server are on different machines. We observe that irrespective of the `-A` flag on the client, the 2MSL wait time is induced. Here the 2MSL wait lasts for 30s. After that, the client attempts to contact the server, which has already exited.

One interesting thing happens if we switch the client and server machines. We will now use Windows as the server and Linux as the client and repeat the experiment:

(start server on Windows machine)

```
C:\> sock -v -s 6666
```

(connect to it from second - Linux - machine)

```
Linux% sock -A -v 192.168.10.145 6666
```

(server identifies incoming connection)

```
connection on 192.168.10.145.6666 from 192.168.10.145.32843
```


(client identifies connection establishment, and is interrupted)

```
connected on 192.168.10.144.32843 to 192.168.10.145.6666
```

```
^C
```

```
Linux%
```

(server identifies connection has terminated and exits)

```
connection closed by peer
```

```
C:\>
```

(client is restarted, specifying same port number as before)

```
Linux% sock -A -b 32843 -v 192.168.10.144 6666
```

```
bind() error: Connection refused
```

At this point we would expect local port 32843 to be unavailable, but because of the way `-A` works on Linux, we are allowed to make use of it. This is a violation of the original TCP specification, but it is allowed by [RFC1122] and [RFC6191], as mentioned before. These specifications allow a new connection request to arrive and be accepted for a connection that is in the `TIME_WAIT` state, if there is a strong reason to believe that segments on the new connection will not be confused with segments on the previous instantiation of the connection based on a combination of the sequence numbers and timestamps. [RFC1337] and the appendix of [RFC1323] show some of the pitfalls related to this rule.

13.5.3 Quiet Time Concept

The 2MSL wait provides protection against delayed segments from an earlier instantiation of a connection being interpreted as part of a new connection that uses the same local and foreign IP addresses and port numbers. But this works only if a host with connections in the 2MSL wait does not crash.

What if a host with connections in the `TIME_WAIT` state crashes, reboots within the MSL, and immediately establishes new connections using the same local and foreign IP addresses and port numbers corresponding to the local connections that were in the `TIME_WAIT` state before the crash? In this scenario, delayed segments from the connections that existed before the crash can be misinterpreted as belonging to the new connections created after the reboot. This can happen regardless of how the initial sequence number is chosen after the reboot.

To protect against this scenario, [RFC0793] states that TCP should wait an amount of time equal to the MSL before creating any new connections after a reboot or crash. This is called the *quiet time*. Few implementations abide by this because most hosts take longer than the MSL to reboot after a crash. Also, if applications use their own checksums or encryption, errors such as these are easily detected.

13.5.4 FIN_WAIT_2 State

In the `FIN_WAIT_2` state, TCP has sent a FIN and the other end has acknowledged it. Unless a half-close is being performed, the TCP must wait for the application on the other end to recognize that it has received an end-of-file notification and close its end of the connection, which causes a FIN to be sent. Only when the application performs this close (and its FIN is received) does the active closing TCP move from the `FIN_WAIT_2` to the `TIME_WAIT` state. This means that one end of the connection can remain in this state forever. The other end is still in the `CLOSE_WAIT` state and can remain there forever, until the application decides to issue its close.

Many implementations prevent this infinite wait in the `FIN_WAIT_2` state as follows: If the application that does the active close does a complete close, not a half-close indicating that it expects to receive data, a timer is set. If the connection is idle when the timer expires, TCP moves the connection into the `CLOSED` state. In Linux, the variable `net.ipv4.tcp_fin_timeout` can be adjusted to control the number of seconds to which the timer is set. Its default value is 60s.

13.5.5 Simultaneous Open and Close Transitions

We have seen the normal uses for the `SYN_SENT` and `SYN_RCVD` states that correspond to sending and receiving SYN segments, respectively. As illustrated in Figure 13-3, TCP was purposely designed to handle simultaneous opens that result in a single connection. When a simultaneous open occurs, the state transitions differ from those shown in Figure 13-9. Both ends send a SYN at about the same time, entering the `SYN_SENT` state. When each end receives its peer's SYN segments, the state changes to `SYN_RCVD`, and each end resends a SYN and acknowledges the received SYN. When each end receives the SYN plus the ACK, the state changes to `ESTABLISHED`.

For a simultaneous close, in terms of Figure 13-6, both ends go from `ESTABLISHED` to `FIN_WAIT_1` when the application issues the close. This causes both FINs to be sent, and they probably pass each other somewhere in the network. When its peer's FIN arrives, each end transitions from `FIN_WAIT_1` to the `CLOSING` state, and each endpoint sends its final ACK. Upon receiving a final ACK, each endpoint's state changes to `TIME_WAIT`, and the 2MSL wait is initiated.

13.6 Reset Segments

We mentioned the *RST* bit field in the TCP header in Chapter 12. A segment having this bit set to “on” is called a “reset segment” or simply a “reset.” In general, a reset is sent by TCP whenever a segment arrives that does not appear to be correct for the referenced connection. (We use the term *referenced connection* to mean the